The Case for Synthesized Memory in Foundation Models

Anonymous ACL submission

Abstract

Foundation models engage with knowledge sources in three ways: they absorb them into pretrained weights, retrieve from them at query time, or re-read them in a long context. None of the three turns a *specific* source into reasoning state that is built once and reused across queries. This position paper argues that this missing operation, which we call *source-conditional memory synthesis* (SCMS), deserves recognition as a fourth mechanism in its own right. We give SCMS a precise functional definition, separate it from a stricter scaling property and from the hypernetwork construction mechanism, and use the resulting taxonomy to organize recent work on generated adapters, distilled KV caches, test-time training, memory tokens, and recurrent state. We then make three arguments, from amortization, compression, and persistence, each stated together with the workloads where it fails. Whether the published results carry over to frontier models and repository-scale sources remains open, and we lay out the tests that would settle the question.

1 Introduction

Picture a coding agent that is assigned to a large software repository and asked thousands of questions about it over a week: where a function lives, how two modules interact, whether a refactor is safe. Today the agent can come to "know" the repository in one of three ways (Figure 1). It can have the repository *trained in*, paying GPU-hours for a snapshot that goes stale with the next commit. It can *retrieve* chunks per query, which is cheap and citable but blind to evidence the retriever fails to surface. Or it can *attend* over the full repository in a long context, paying a length-dependent cost on every query and holding a large KV cache.

Each option lands in a different corner of a cost surface, and one corner stays empty. That corner has moderate one-time encoding, reusable state, and no source-length-dependent term at query time. A growing body of work now aims at it. In these methods an operator reads a source once and emits compact state that the base model consumes directly, such as LoRA weights, a KV prefix, memory tokens, or a recurrent state, and later queries reuse that state. The idea descends from hypernetworks (Ha et al., 2017) and fast weights (Schmidhuber, 1992; Schlag et al., 2021). What has changed since 2024 is the evidence: several unaffiliated groups have shown that such state is useful at document scale, and that it can sometimes be produced in a single forward pass (Charakorn et al., 2026; Liu et al., 2026; Eyuboglu et al., 2025).

> **Position.** Source-conditional memory synthesis should be recognized as a first-class, *fourth* mechanism for engaging foundation models with persistent knowledge, alongside pretraining, retrieval, and long context. It complements these three rather than replacing them, and the field should organize foundational research around it.

We make four contributions: (i) a definition that separates the shared source-to-state operation from the properties particular methods do or do not achieve, together with a taxonomy of existing work (§3); (ii) three arguments, each scoped to the regime in which it holds (§4); (iii) the strongest alternative views, with falsification criteria (§6); and (iv) a research agenda and evaluation protocol (§7). Throughout, we keep *established results*, *preliminary evidence*, and *hypotheses* apart.

2 Three Walls

Parametric memory. Storing knowledge in weights is the default, and it is what scaling laws describe (Hoffmann et al., 2022). It is also monolithic and stale. Every fact must be present at training time, and every update requires more optimization. The rise of knowledge editing (Meng et al., 2022, 2023; Park et al., 2025) is a response to this mismatch between static weights and knowledge that changes. Parametric memory also entangles

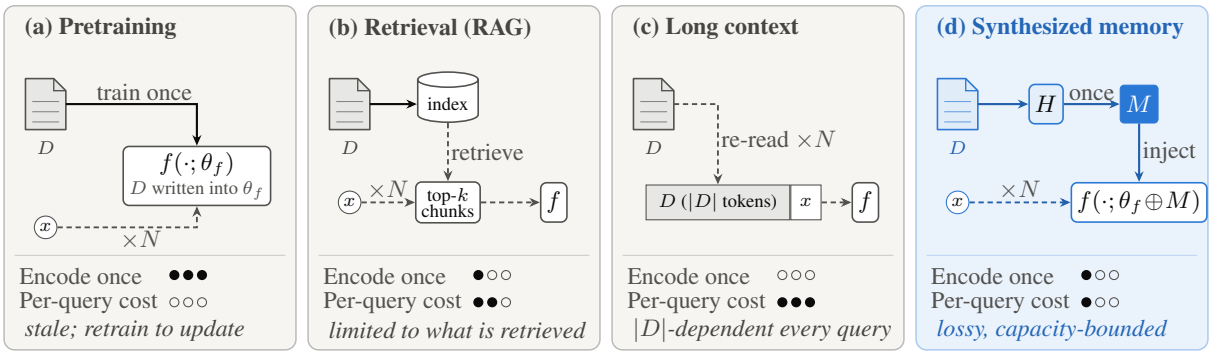


Figure 1: **Four ways to give a model access to a source D.** Solid arrows are paid once per source version; dashed arrows are paid on each of N queries. k is the number of chunks RAG retrieves per query. Meters give qualitative encoding and per-query costs; the bottom line names each mechanism's characteristic weakness. In (d), a memory operator H encodes D into state M once, and M is injected into the frozen model for every later query. Encoding in (d) costs from one forward pass to hours of optimization, depending on H (Table 1).

general capability (reasoning, instruction following) with source-specific knowledge (this repository, this user), which ought to be separable.

Retrieval. Retrieval-augmented generation (RAG; Lewis et al., 2020) moves knowledge into an external index. It is the right tool for many workloads, especially sparse exact lookup and attribution. Its limit is structural. In a retrieve-then-condition pipeline, evidence that is not retrieved never reaches the reader, and the retrieval decision is made before the model knows what reasoning it will need. NovelHopQA (Gupta et al., 2025) documents the consequence: accuracy falls as hops and context length grow, often because final-hop evidence is missed.

Long context. Context windows now reach a million tokens or more. Prefilling a source of length $|D|$ costs $O(|D|^2)$ attention. Prefix caching amortizes the prefill, but it still leaves an $O(|D|)$ attention term for every generated token and a KV cache that grows with $|D|$ and must stay resident in memory. Accuracy also depends on where information sits in the window (Liu et al., 2024a). When the same source is consulted many times, every one of these costs recurs in full.

The shared gap. The three mechanisms fail in different ways, but they lack the same operation. None of them treats *encoding a specific source into reusable reasoning state* as a learnable step that runs once per source version and is reused across queries. Pretraining encodes too early and too coarsely, RAG never encodes the source into model state, and long context re-reads it every session. Beyond this shared gap, we do not claim the three are equivalent.

Why now. The gap matters more today than it did a few years ago. Agents now return to the same repositories, users, and documents across many sessions, so reuse is the common case rather than the exception. At the same time, holding long KV caches in accelerator memory limits how many long-context requests a server can keep in flight. Both trends favor state that is built once, kept small, and reused across many queries.

3 Source-Conditional Memory Synthesis

3.1 Definition and terminology

Let $f(\cdot;\theta_f)$ be a base model and D a *source*, such as a document, repository, user history, or trajectory. A *memory operator* H maps the source to a memory state that is injected into the model:

$$M = H(D;\theta_H), \qquad y = f(x;\theta_f \oplus M). \quad (1)$$

Here $\oplus$ conditions the model without modifying θ_f. It can mean adding LoRA matrices, prepending prefix or KV states or memory tokens, or initializing a recurrent state. Three notions are often conflated in this area, and we keep them apart:

- **SCMS** is the *functional class*. D is processed once per source version into model-consumable state M, and later queries reuse M *without reprocessing D and without query-time access to a source-sized external store*. The construction episode itself may be iterative.
- **SOCI** (synthesize once, corpus-size independent) is a *stricter scaling property*. Both the size of M and the per-query cost attributable to M must be independent of $|D|$, although one-time construction may still scale with $|D|$.
- A **proper hypernetwork** is a *construction mechanism*: a learned network whose forward pass

emits parameters, parameter deltas, or adapters for another network (Ha et al., 2017).

These facets are orthogonal. We use "hypernetwork" only in the third sense. SCMS says what methods share. It does not promise any particular construction time, state size, fidelity, or composition rule; these must be assessed per method.

Training and refreshing H. A learned operator is trained *across a distribution of sources*, not on the deployment source. Its objective is that $f(\cdot; \theta_f \oplus H(D))$ match a teacher that sees D in context. The training triples do not require annotation. Following self-study (Eyuboglu et al., 2025) and context distillation (Charakorn et al., 2026), one (1) collects unlabeled sources, (2) has the base model generate diverse queries about each source, and (3) labels those queries with the base model conditioned on the full source. Query volume then scales without human effort, at the price of teacher generation and with no guarantee of covering the deployment query distribution. Logged deployment queries can be mixed in when available.

When a source changes to D', a *full refresh* reruns the operator, $M' = H(D'; \theta_H)$. It needs no retraining of θ_H or change to θ_f, but it rereads D'. An *incremental refresh*, $M' = U(M, \Delta D)$, would skip unchanged content but must control accumulated error; it remains open (§7.4).

3.2 Existing methods through this lens

Table 1 classifies existing work by state, construction mechanism, and the two stricter properties. Figure 2 maps them by construction time.

Forward-generated adapters. Doc-to-LoRA (Charakorn et al., 2026) trains a ∼309M-parameter Perceiver-style encoder that emits a rank-8 LoRA for Gemma-2-2B (Gemma Team, 2024) in a sub-second forward pass. For a 128K-token document the adapter is under 50 MB, against more than 12 GB of uncompressed KV cache. Text-to-LoRA (Charakorn et al., 2025) conditions on task descriptions instead, and SHINE (Liu et al., 2026) maps a context to a LoRA in a single pass. These are the only families in Table 1 whose construction is a proper hypernetwork.

Optimized adapters and states. LoRA (Hu et al., 2022) specifies how state is *represented*, not how it is *constructed*. A per-source LoRA obtained by optimization shows that an adapter can carry reusable, source-specific information. It is not evidence of low construction latency, and it can degrade base-model quality. Test-time training (Hu et al., 2025; Tandon et al., 2025; Bansal et al., 2025) builds source-specific weights with gradient steps at inference time, while Cartridges (Eyuboglu et al., 2025; Diaz, 2025) and prefix tuning (Li and Liang, 2021) optimize a KV or prefix state for each source.

Memory tokens. Learned tokens that are not conditioned on D are not SCMS. Source-conditioned tokens, such as gist tokens, AutoCompressors, and ICAE (Mu et al., 2023; Chevalier et al., 2023; Ge et al., 2024), do qualify. They replace attention over $|D|$ raw tokens with attention over m synthesized ones. With a fixed $m \ll |D|$, the source-attributable cost per query is bounded, but the state is lossy. If m grows with the source or with an agent's history, storage and attention grow with it. Growing memory tokens are therefore compressed context, not a fixed-size answer to context growth.

Linear attention and state-space models. Linear attention and SSMs (Katharopoulos et al., 2020; Gu et al., 2022; Gu and Dao, 2023) compress a sequence into a fixed recurrent state, and linear attention is formally a fast-weight programmer (Schlag et al., 2021). A state that summarizes D and is *checkpointed and reused* across independent queries is SCMS. It is also SOCI as long as no growing cache accompanies it. Rescanning D for every query is efficient long-context processing, not encode-once memory, and a hybrid that keeps a full-attention cache in some layers is not strictly SOCI. Most evidence for these architectures concerns continuous sequences, so checkpoint-and-branch reuse still needs direct evaluation.

Boundary and adjacent cases. Knowledge editing (Meng et al., 2022, 2023) maps a single fact to a weight update (analytically, in ROME). It is a boundary case of the abstraction, and the family as a whole is not uniformly SCMS. Mixture-of-LoRAs systems (Kong et al., 2024; Qiu et al., 2026; Liao et al., 2025; Zeng et al., 2025) *select* existing adapters instead of synthesizing new state, and RAG queries a source-sized store every time. Both are adjacent to SCMS, not instances of it.

3.3 Representation versus construction

As Figure 2 illustrates, reported construction costs for methods that share a representation span about five orders of magnitude, so we read the evidence in two separate parts. *Optimization-based methods*

Method family	State M	Construction	HN	SOCI if	Principal limitations
FORWARD SYNTHESIS					
Doc-to-LoRA, Text-to-LoRA, SHINE	Generated LoRA or weight deltas	Hypernetwork pass	✓	budget fixed	Budget bounds fidelity; rank can grow with $\|D\|$; untested at scale
Source-conditioned memory tokens	m synthesized tokens	Learned encoder	–	m fixed	Fixed m is lossy; growing m grows attention and storage
OPTIMIZATION					
LoRA fine-tuning, TTT	Optimized adapters or weights	Gradient steps	–	state reused	Seconds to hours per source; interference; base-quality loss
Cartridges, prefix tuning	Optimized KV or prefix	Self-study, distillation	–	state reused	Minutes to hours per source; prefix length bounds capacity
RECURRENCE AND EDITING					
Linear attention, SSMs	Recurrent state	Recurrence	–	checkpointed	Fixed capacity; reuse across queries rarely evaluated
Knowledge editing	Localized weight deltas	Analytic or optimized	–	few edits	Single facts; interference accumulates with edits

Table 1: **A taxonomy of source-to-state methods**, grouped by how the state is constructed. Being a proper hypernetwork (HN) and being strictly SOCI are separate properties that are stricter than SCMS, and the SOCI column gives the condition each family must meet. Plain prompting, routing over a fixed adapter library, and RAG are related to SCMS but are not instances of it. Members of a family share the SCMS operation, but they need not share a mechanism, a cost profile, or a level of quality, and each must be assessed on its own.

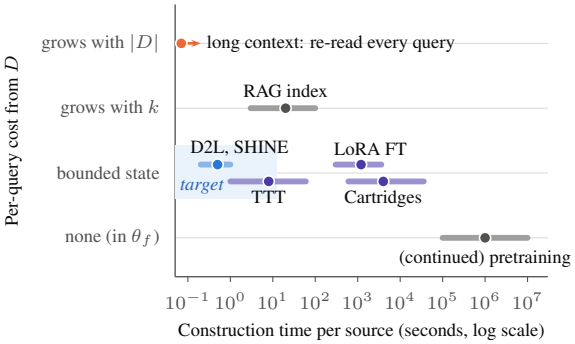


Figure 2: **Design space of source-to-state mechanisms.** Methods are placed by construction time per source (dots mark typical values and bars reported ranges, both order-of-magnitude and hardware-dependent) and by the per-query cost attributable to D. Color marks construction: ● forward pass, ● optimization, ● other. Only forward synthesis currently reaches the shaded *target* of low construction latency with bounded state.

show that compact source-specific states can support strong adaptation in some regimes. Forward-pass hypernetworks show that useful states can be synthesized at low latency in narrower evaluated regimes. Whether forward synthesis can match the quality and scale of optimization-based adaptation remains open. We therefore never cite fine-tuning or TTT results as evidence that a proper hypernetwork reaches the same quality. When we discuss latency, we mean forward synthesis. For quality claims, we name the mechanism that produced the evidence and the setting in which it was measured.

The regime we target is *reuse-heavy, source-specific state whose construction latency matters*. Frequently updated sources are an important instance, but they are not the only one. Forward synthesis also pays off for stable sources when a system must build and serve many repository-, user-, or task-specific states, or when per-source optimization infrastructure is not available.

4 Three Arguments

4.1 The amortization argument

Suppose a source D is queried N times. With a one-time encoding cost $C_{\text{enc}}(D)$ and a per-query cost C_{q}, the total cost is

$$C_{\text{total}}(N) = C_{\text{enc}}(D) + N\,\mathbb{E}_x[C_{\text{q}}(D, x)]. \quad (2)$$

For synthesized memory, C_{enc} is the cost of running H, which is reported as sub-second for Doc-to-LoRA on Gemma-2-2B and as minutes to hours of optimization for Cartridges. An operator whose output can depend on arbitrary content of D must read that content. So C_{enc} generally scales with $|D|$, and synthesis is neither free nor sublinear. The advantage it offers lies in the second term.

Regime-conditioned comparisons. Synthesis does not dominate any alternative unconditionally. The comparisons below assume matched quality.

Versus long context. Synthesis is cheaper once

$$N > N^* = \frac{C_{\text{enc}}^{\text{syn}}}{C_{\text{q}}^{\text{LC}} - C_{\text{q}}^{\text{syn}}}, \quad (3)$$

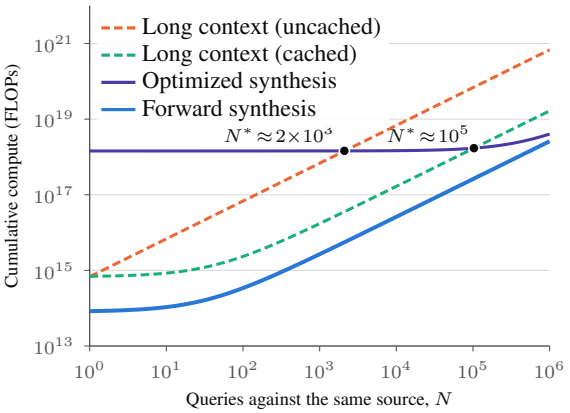


Figure 3: **Cumulative compute for one 128K-token source**, from back-of-the-envelope FLOP estimates for a Gemma-2-2B-class model with ∼500 query and response tokens (§4.1). Both synthesis curves assume matched quality and omit adapter loading. Dots mark the break-even points of optimized synthesis against the two long-context baselines.

where $C_{\text{q}}^{\text{syn}}$ includes forward, adapter-loading, and routing costs. Figure 3 gives rough estimates for a 128K-token source and a model with $P \approx 2.6$B parameters. Uncached prefill costs at least $2P|D| \approx 6.8 \times 10^{14}$ FLOPs per query, while Doc-to-LoRA's 309M-parameter encoder reads the source once for about 8×10^{13} FLOPs, so forward synthesis breaks even on the first query. With a prefix cache, the $|D|$-dependent term falls to about 1.4×10^{13} FLOPs per query, and the break-even point moves to a handful of queries. If a Cartridge costs one GPU-hour (about 1.4×10^{18} FLOPs), it needs roughly 2×10^3 queries against uncached long context and 10^5 against a cached one. Residency matters as much as compute: the uncompressed KV cache for this source exceeds 12 GB, whereas the generated adapter stays under 50 MB. These are estimates, not measurements, and real values depend on hardware and serving overhead.

Versus RAG. Consider reuse-heavy workloads that require dense or global interactions across a source. There, synthesized state can amortize query-time source access and avoid repeated evidence selection. It does not remove the need for multi-step reasoning, which a multi-hop query still requires, and it does not guarantee that every required fact survives compression. Retrieval remains preferable for sparse exact lookup, attribution, and provenance, and whenever the synthesized state is not faithful enough. Against single-round RAG with small k, the compute difference is small, so cost alone does not justify synthesis.

Versus pretraining. Here the distinction is operational, not asymptotic. Once H is trained, it produces detachable state for a new source without touching θ_f and without per-source optimization. Pretraining remains the right home for stable, globally shared knowledge. Synthesis targets three kinds of source: those introduced or updated after base-model training, those that should stay separable, and those reused across many queries.

Worked example (motivating, not demonstrated). Suppose a coding agent works on a repository with a ∼200K-token working set. It issues ∼10K queries a week for 26 weeks, and the repository changes daily. Long context pays a $|D|$-dependent term on ∼260K queries. Synthesis pays for ∼180 refreshes plus per-query costs with no such term, *provided* a synthesizer of adequate fidelity exists at this scale. Current evidence does not yet establish that proviso (§7.5).

Systems and training costs. A deployment comparison must count every term in a common unit over a fixed horizon:

$$C = C_{\text{train}}^{H} + \textstyle\sum_s C_{\text{enc}}(D_s) + \sum_{\text{queries}} C_{\text{q}} + C_{\text{load}} + C_{\text{route}} + C_{\text{refresh}}. \quad (4)$$

Resident memory enters separately, as a capacity constraint. C_{load} depends on the residency and eviction policy. S-LoRA (Sheng et al., 2024) studies this cost for static adapters, but no serving infrastructure of that kind yet exists for synthesized state. Routing reduces to a lookup when the source's identity is known, and composing several memories is open (§7.3). A learned synthesizer shared by S sources, each queried $\bar{N}$ times, is worth training when

$$S\bar{N}\,(C_{\text{q}}^{\text{alt}} - C_{\text{q}}^{\text{syn}}) > C_{\text{train}}^{H} + S\,C_{\text{enc}}^{\text{syn}}. \quad (5)$$

No published work reports the end-to-end cost of training a synthesizer. A base-model upgrade may also force retraining H and re-synthesizing every retained source. Hence no universal threshold exists, and the guidance is workload-dependent. Sources queried only a few times favor long context or RAG. Optimization-based synthesis pays off only after heavy reuse. Forward synthesis is most attractive in three cases: one synthesizer serves many sources, each source is queried many times, or resident KV memory is the binding constraint. For scale, Cartridges report 38.6× less memory and 26.4× higher throughput than in-context learning at matched quality (Eyuboglu et al., 2025), and

KV compaction reaches 20–40× with little accuracy loss (Staniszewski and Łańcucki, 2025).

4.2 The compression argument

The amortization argument is economic. The compression argument concerns sample efficiency, and we state it as a structured analogy that yields a testable prediction, not as a theorem at LLM scale.

A model that has only parametric memory is an instance-level learner, which must store in θ_f whatever it will later be asked. A model with a learned synthesis pathway is a meta-learner in which sources play the role of tasks. Under this correspondence, a *task* is a source $D \sim \mathcal{T}$ drawn i.i.d. from a domain, the *meta-parameters* are θ_H, and *per-task adaptation* is $M = H(D; \theta_H)$. Environment-level and PAC-Bayes lifelong-learning bounds (Baxter, 2000; Pentina and Lampert, 2014; Amit and Meir, 2018) share a common structure. Given S training sources with m query–answer pairs each, the risk on a new source is bounded, schematically, by

$$\hat{R} + O\Big(\sqrt{\tfrac{\ell(M|\theta_H)+\log(m/\delta)}{m}}\Big) + O\Big(\sqrt{\tfrac{\ell(\theta_H)+\log(S/\delta)}{S}}\Big). \tag{6}$$

Here $\ell(M \mid \theta_H)$ is the description length of the source-specific state relative to the prior that the learned synthesizer induces (a KL term in the PAC-Bayes form), and $\ell(\theta_H)$ is the complexity of the shared prior. The first term is paid per source. The second is paid once and amortized over all sources. An instance-level learner instead pays $\ell(M \mid P_0)$ under a fixed, uninformed prior P_0.

Prediction. When sources share regularities that a learned prior can capture, $\ell(M \mid \theta_H) \ll \ell(M \mid P_0)$, and synthesis should reach a better efficiency–quality trade-off at fixed compute. Domains with strong shared structure, such as codebases, legal corpora, and clinical records, should therefore gain more than heterogeneous domains, which cross-domain comparisons can test directly. The supporting evidence so far is consistent but indirect. In implicit neural representations, hypernetwork quality improves with encoder scale (Gu and Yeung-Levy, 2025), and Doc-to-LoRA's zero-shot cross-modal transfer points to a reusable learned prior. Scaling laws currently cover only parameters and tokens for unconditional models (Hoffmann et al., 2022). If the prediction holds, the capacity devoted to conditional synthesis is a missing scaling axis.

4.3 The persistence argument

Agents return to the same environment, repository, or user many times, and each existing mechanism falls short in its own way. In-context adaptation does not survive the session. RAG surfaces evidence but leaves the model's state unchanged from one query to the next. Fine-tuning by optimization does integrate information, but each update takes seconds to hours, which rules out high-frequency updates inside interactive loops. All three provide useful state. What none provides is state that is at once persistent, integrated into the model's computation, and cheap to construct and inject.

We treat SCMS as a *common interface through which four properties can be pursued jointly*, not as a mechanism that already guarantees them:

- **integration**: the source changes the model's computation, not just its prompt;
- **low construction latency**: so far reached only by forward synthesis in evaluated settings;
- **persistence**: the state survives across sessions and can be versioned;
- **compositionality**: several memories combine at inference time without retraining.

The evidence for compositionality is thin. Cartridges compose without retraining in their reported settings, while the adapter-merging literature documents interference. No cited method yet demonstrates all four properties at scale, and Table 1 exists to make that gap visible method by method.

To make the interface concrete, consider the coding agent again. At the start of each session it loads versioned memories for the repository and for the user; after each merged change it re-synthesizes only the repository memory, and it falls back to retrieval when an answer must cite code.

On the WebArena multi-site split, grounded test-time adaptation (Chen et al., 2025) raises success from 2% to 23% by learning environment dynamics during deployment. Its mechanism keeps the learned world model *in context*. We therefore count it as adjacent evidence that reusable environment representations help, not as SCMS.

5 Convergence

Three largely independent streams have arrived at related source-to-state interfaces:

- **forward-pass synthesis**: Doc-to-LoRA, Text-to-LoRA, SHINE, and related hypernetworks (Charakorn et al., 2026, 2025; Liu et al., 2026; Gu and Yeung-Levy, 2025);

- **KV-cache and prefix synthesis**: Cartridges and follow-ups, KVSculpt, transform coding, and CacheGen (Eyuboglu et al., 2025; Diaz, 2025; Jiang and Jin, 2026; Staniszewski and Łańcucki, 2025; Liu et al., 2024b);
- **test-time training and recurrent state**: TTL, TTT-E2E, qTTT, and state-space models (Hu et al., 2025; Tandon et al., 2025; Bansal et al., 2025; Gu and Dao, 2023).

The convergence is *functional*. The streams implement related operations, but they do not share a mechanism, identical asymptotic scaling, or demonstrated equal performance. What the convergence does show is that cross-stream questions, such as composition, pretraining for synthesizability, and synthesizer scaling, are under-studied because the streams have not been viewed together.

6 Alternative Views

"Synthesis is a useful systems primitive, not a new mechanism." This is the strongest alternative, and we agree with much of it. Synthesized state should complement retrieval, long context, and fine-tuning, and which one to use should depend on cost, update rate, and auditability. We part ways on a single point. The primitive view treats synthesis as an optimization layered on top of unchanged models. We argue that it also warrants *foundational* research the primitive view would not motivate: pretraining for synthesizability, scaling laws for synthesizer capacity, and the semantics of composition (§7). If that research yields nothing beyond what engineering alone delivers, the primitive view is right.

"RAG, or long context, is enough." Each alternative has workloads it serves best. RAG is preferable when evidence is sparse and localizable, when citation and provenance are required, or when a source is rarely reused. Long context suits one-off access, and fine-tuning suits stable knowledge. Iterative, graph-based, and co-adapted retrieval mitigate the retrieve-then-condition limit (Gupta et al., 2025) but do not remove it. Our claim is narrower than "RAG is insufficient." On reuse-heavy workloads with dense interactions across a source, synthesis trades the per-query retrieval or attention bottleneck for an *encoding-and-compression* bottleneck. We expect hybrids to outperform either component alone. In such a hybrid, M carries reusable structure while retrieval over the retained source supplies exact and attributable facts.

"Mixture-of-experts already does this." MoE (Shazeer et al., 2017) and Mixture-of-LoRAs route inputs among a fixed library of trained experts. Synthesis differs in granularity (it conditions per source, not per token), in coverage (it handles new sources, not a finite library), and in composition (it creates state instead of selecting it). The two approaches are complementary, and a system could well route over synthesized memories.

"Hypernetworks don't scale" and "this is fine-tuning rebranded." Hypernetworks have a history of training difficulties (Chauhan et al., 2024). Recent document-scale results update this prior without settling it, because frontier-scale evidence is absent. Forward synthesis differs from fine-tuning in latency (a forward pass instead of an optimization loop), in mechanism (it is non-iterative), and in its prior (a learned θ_H instead of an uninformed initialization). These differences concern construction cost, not quality (§3.3).

What would falsify the position. The position makes three predictions:

- forward-synthesized memory stays competitive with in-context learning as base models scale;
- composed memories interfere only mildly;
- on reuse-heavy workloads, synthesis gives practical advantages under matched quality, provenance, and systems cost.

Failure on any of these counts against the corresponding argument. If RAG or long-context systems improve enough in cost and reliability to erase the amortization advantage on reuse-heavy workloads, the case for a distinct mechanism weakens.

7 A Research Agenda

Table 2 condenses the agenda into proposed experiments, each paired with the outcome that would count against the position.

7.1 Pretraining for synthesizability

Today's synthesizers adapt off-the-shelf models whose pretraining never rewarded amenability to injected state. Candidate objectives include auxiliary losses on the behavior of adapter-modified copies, gisting-style bottlenecks (Mu et al., 2023), and inner-loop meta-learning objectives that simulate synthesis during pretraining. Whether any of these objectives improves synthesized memory at fixed compute, and at what scale, is an open question for controlled pretraining studies.

Question	Proposed test	Fails if
Pretraining §7.1	Add a synthesizability loss at fixed compute	no gain over the baseline
Scale §7.2	Sweep model size vs. ICL at fixed $\|D\|$	the gap widens with scale
Composition §7.3	Merge k memories and track retention	interference grows fast in k
Updating §7.4	Run $U(M, \Delta D)$ over edit streams	error drifts unless D is re-read
Capacity §7.5	Grow $\|D\|$ at a fixed budget	quality fails at practical $\|D\|$

Table 2: **Proposed experiments.** Each row pairs an open question with a test and the outcome that would count against the position.

7.2 Scaling laws and the model-scale test

The evidence is concentrated at small-to-mid scale. That scale is itself a relevant deployment regime for cost-sensitive, on-device, and multi-tenant serving, but it does not establish frontier behavior. The decisive experiment is a model-scale sweep that compares forward-synthesized memory with in-context learning at matched source length. It should report how the quality gap and the stability of synthesizer training change with base-model size.

7.3 Compositionality

A coding agent should be able to combine a repository memory with a style-guide memory. It is open when composition preserves information, how to design state that composes additively, whether H can be trained with composition as an explicit objective, and how to control adapter interference.

7.4 Online synthesis and incremental updates

A deployment has to decide when to re-synthesize and when to reuse, how to evict and version memories, and how to expose synthesis as a managed serving primitive alongside KV management. The central algorithmic problem is incremental updating, $M' = U(M, \Delta D)$, with bounded accumulated error. Existing proper hypernetworks demonstrate fast full synthesis only in their evaluated settings, and streaming updates remain undemonstrated. A simple first policy re-synthesizes a versioned memory once the source diff exceeds a threshold.

7.5 Capacity, reliability, and provenance

A fixed-size state cannot preserve arbitrary exact access to an unbounded source, so practical capacity depends on the query distribution. Current proper-hypernetwork evidence sits at document or context scale; for example, Doc-to-LoRA reports near-perfect synthetic needle retrieval beyond 32K tokens. Repository-scale, million-token, and continually accumulated memories are targets, not demonstrated operating points. A retrieval miss can be recovered on the next query, but information lost during synthesis stays lost until M is refreshed, enlarged, or supplemented. Three remedies follow: keep the raw source as a cold path, tune the state budget, and develop detectors that flag when M is insufficient and trigger retrieval or abstention.

7.6 An evaluation protocol

Because the evidence is spread across papers that differ in model, task, and baseline, we propose shared reporting conventions:

1. report three quantities separately: source length ingested, state size and its scaling, and source length at which quality is measured;
2. evaluate *checkpoint-and-branch* reuse, where independent queries share one state, not only continuous sequences;
3. compare mechanisms at matched quality, in one cost unit covering loading and refresh;
4. report refresh fidelity after source edits.

Existing long-context and multi-hop suites can be rerun under these conventions, so no new benchmark is needed to start comparing the three streams.

8 Conclusion

Pretraining, retrieval, and long context all lack one operation: turning a specific source into reasoning state that can be reused. Source-conditional memory synthesis is our name for that operation. The methods that implement it differ in representation, construction cost, capacity, and maturity, and none of them yet combines integration, low latency, persistence, and composition at scale. We have argued that the operation deserves to be treated as a fourth mechanism with its own research program, and we have said what evidence would show that we are wrong. The next steps are concrete. Systems researchers can build serving infrastructure for versioned, composable state. Scaling researchers can run the model-scale test of §7.2. Agent researchers can build loops that refresh memory instead of rereading the source. Progress on any of these will show whether synthesized memory earns its place next to pretraining, retrieval, and long context.

Limitations

Our evidence draws on several research lines whose models, tasks, and baselines differ, so cross-paper numbers should be interpreted within their respective experimental settings. The cost analysis is intended to illustrate the relevant trade-offs rather than provide an end-to-end systems measurement. Existing evidence for forward synthesis is concentrated in models with a few billion parameters and sources of up to roughly 10^5 tokens, leaving frontier-scale quality, synthesizer training cost, and training stability as important open questions. The compression argument is motivated by connections to meta-learning bounds and yields testable predictions, rather than constituting a theorem for LLMs. Synthesized memory is inherently capacity-bounded and does not currently provide token-level attribution, which motivates complementary retrieval when exact recall or provenance is required.

Ethical Considerations

Synthesized memory introduces risks that differ from those of ordinary token context:

- *Privacy.* Deleting a tenant-scoped M removes the deployed source-specific state, but does not guarantee removal of information that may persist in logs, caches, prior outputs, or the synthesizer itself. M may also reveal information about its source under probing, motivating membership-inference and extraction evaluations.
- *Adversarial sources.* A poisoned source D may induce persistent behavioral changes through synthesized state. Practical safeguards include provenance-aware synthesis and behavioral validation before a memory is admitted for reuse.
- *Staleness.* Each memory should be associated with a specific source version so that stale states can be invalidated and downstream outputs traced to the version that produced them.
- *Provenance.* In settings where citation or auditability is required, synthesized memory should be complemented with access to the underlying source.

References

Ron Amit and Ron Meir. 2018. Meta-Learning by adjusting priors based on extended PAC-Bayes theory. In *Proceedings of the 35th International Conference on Machine Learning*.

Rachit Bansal, Aston Zhang, Rishabh Tiwari, Lovish Madaan, Sai Surya Duvvuri, Devvrit Khatri, David Brandfonbrener, David Alvarez-Melis, Prajjwal Bhargava, Mihir Sanjay Kale, and Samy Jelassi. 2025. Let's (Not) just put things in context: Test-Time training for Long-Context LLMs. arXiv preprint arXiv:2512.13898.

Jonathan Baxter. 2000. A model of inductive bias learning. *Journal of Artificial Intelligence Research*, 12:149–198.

Rujikorn Charakorn, Edoardo Cetin, Yujin Tang, and Robert Tjarko Lange. 2025. Text-to-LoRA: Instant Transformer adaption. arXiv preprint arXiv:2506.06105.

Rujikorn Charakorn, Edoardo Cetin, Shinnosuke Uesaka, and Robert Tjarko Lange. 2026. Doc-to-LoRA: Learning to instantly internalize contexts. arXiv preprint arXiv:2602.15902.

Vinod Kumar Chauhan, Jiandong Zhou, Ping Lu, Soheila Molaei, and David A. Clifton. 2024. A brief review of hypernetworks in deep learning. *Artificial Intelligence Review*, 57(9).

Arthur Chen, Zuxin Liu, Jianguo Zhang, Akshara Prabhakar, Zhiwei Liu, Shelby Heinecke, Silvio Savarese, Victor Zhong, and Caiming Xiong. 2025. Grounded Test-Time adaptation for LLM agents. arXiv preprint arXiv:2511.04847.

Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Maurizio Diaz. 2025. Learned structure in Cartridges: Keys as shareable routers in Self-Studied representations. arXiv preprint arXiv:2508.17032.

Sabri Eyuboglu, Ryan Ehrlich, Simran Arora, Neel Guha, Dylan Zinsley, Emily Liu, Will Tennien, Atri Rudra, James Zou, Azalia Mirhoseini, and Christopher Ré. 2025. Cartridges: Lightweight and General-Purpose long context representations via Self-Study. arXiv preprint arXiv:2506.06266.

Tao Ge, Jing Hu, Lei Wang, Xun Wang, Si-Qing Chen, and Furu Wei. 2024. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations*.

Gemma Team. 2024. Gemma 2: Improving open language models at a practical size. arXiv preprint arXiv:2408.00118.

Albert Gu and Tri Dao. 2023. Mamba: Linear-Time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752.

Albert Gu, Karan Goel, and Christopher Ré. 2022. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*.

Jeffrey Gu and Serena Yeung-Levy. 2025. Foundation models secretly understand neural network weights: Enhancing hypernetwork architectures with foundation models. arXiv preprint arXiv:2503.00838.

Abhay Gupta, Kevin Zhu, Vasu Sharma, Sean O'Brien, and Michael Lu. 2025. NovelHopQA: Diagnosing Multi-Hop reasoning failures in long narrative contexts. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*.

David Ha, Andrew Dai, and Quoc V. Le. 2017. HyperNetworks. In *International Conference on Learning Representations*.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, and 3 others. 2022. Training Compute-Optimal large language models. In *Advances in Neural Information Processing Systems*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank adaptation of large language models. In *International Conference on Learning Representations*.

Jinwu Hu, Zhitian Zhang, Guohao Chen, Xutao Wen, Chao Shuai, Wei Luo, Bin Xiao, Yuanqing Li, and Mingkui Tan. 2025. Test-Time learning for large language models. arXiv preprint arXiv:2505.20633.

Bo Jiang and Sian Jin. 2026. KVSculpt: KV cache compression as distillation. arXiv preprint arXiv:2603.27819.

Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. 2020. Transformers are RNNs: Fast autoregressive Transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*.

Rui Kong, Qiyang Li, Xinyu Fang, Qingtian Feng, Qingfeng He, Yazhu Dong, Weijun Wang, Yuanchun Li, Linghe Kong, and Yunxin Liu. 2024. LoRA-Switch: Boosting the efficiency of dynamic LLM adapters via System-Algorithm co-design. arXiv preprint arXiv:2405.17741.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented generation for Knowledge-Intensive NLP tasks. In *Advances in Neural Information Processing Systems*.

Xiang Lisa Li and Percy Liang. 2021. Prefix-Tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*.

Mengqi Liao, Wei Chen, Junfeng Shen, Shengnan Guo, and Huaiyu Wan. 2025. HMoRA: Making LLMs more effective with hierarchical mixture of LoRA experts. In *International Conference on Learning Representations*.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*.

Yewei Liu, Xiyuan Wang, Yansheng Mao, Yoav Gelberg, Haggai Maron, and Muhan Zhang. 2026. SHINE: A scalable In-Context hypernetwork for mapping context to LoRA in a single pass. arXiv preprint arXiv:2602.06358.

Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024b. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2023. Mass-Editing memory in a Transformer. In *International Conference on Learning Representations*.

Jesse Mu, Xiang Lisa Li, and Noah Goodman. 2023. Learning to compress prompts with gist tokens. In *Advances in Neural Information Processing Systems*.

Seongsik Park, Sangmin Park, Jaieun Kim, and Harksoo Kim. 2025. MAKE: Memory-Associated knowledge editing. *Transactions of the Association for Computational Linguistics*.

Anastasia Pentina and Christoph H. Lampert. 2014. A PAC-Bayesian bound for lifelong learning. In *Proceedings of the 31st International Conference on Machine Learning*.

Ruizhong Qiu, Hanqing Zeng, Yinglong Xia, Yiwen Meng, Ren Chen, Jiarui Feng, Dongqi Fu, Qifan Wang, Jiayi Liu, Jun Xiao, Xiangjun Fan, Benyu Zhang, Hong Li, Zhining Liu, Hyunsik Yoo, Zhichen Zeng, Tianxin Wei, and Hanghang Tong. 2026. ReMix: Reinforcement routing for mixtures of LoRAs in LLM finetuning. arXiv preprint arXiv:2603.10160.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. 2021. Linear Transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*.

Jürgen Schmidhuber. 1992. Learning to control Fast-Weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 4(1):131–139.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The Sparsely-Gated Mixture-of-Experts layer. In *International Conference on Learning Representations*.

Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. 2024. S-LoRA: Serving thousands of concurrent LoRA adapters. arXiv preprint arXiv:2311.03285.

Konrad Staniszewski and Adrian Łańcucki. 2025. KV cache transform coding for compact storage in LLM inference. arXiv preprint arXiv:2511.01815.

Arnuv Tandon, Karan Dalal, Xinhao Li, Daniel Koceja, Marcel Rød, Sam Buchanan, Xiaolong Wang, Jure Leskovec, Sanmi Koyejo, Tatsunori Hashimoto, Carlos Guestrin, Jed McCaleb, Yejin Choi, and Yu Sun. 2025. End-to-End Test-Time training for long context. arXiv preprint arXiv:2512.23675.

Hanqing Zeng, Yinglong Xia, Zhuokai Zhao, Chuan Jiang, Qiang Zhang, Jiayi Liu, Qunshu Zhang, Lizhu Zhang, Xiangjun Fan, and Benyu Zhang. 2025. S'MoRE: Structural mixture of residual experts for Parameter-Efficient LLM fine-tuning. arXiv preprint arXiv:2504.06426.